

Design Once, Adapt Efficiently: Multi-Agent System Generation via Optimized Pools

Anonymous ACL submission

Abstract

Large language model (LLM)-based multi-agent systems (MAS) have demonstrated remarkable capabilities across diverse domains, yet their automated design faces a fundamental tension between efficiency and adaptivity. Existing methods either produce a single fixed MAS for all queries, failing to account for query diversity; or re-optimize all components from scratch per query, incurring prohibitive cost and unstable quality. To resolve this dilemma, we propose **OSMAS**, a framework that transforms MAS design from “re-search every time” into “build once, compose efficiently.” OSMAS constructs two reusable, pre-optimized repositories—an *Agent Pool* and a *Topology Pool*—through a decoupled three-phase training paradigm that separately optimizes agent profiles, prompts, and topology. At inference time, a lightweight orchestrator composes a query-specific MAS by retrieving and adapting exemplars from these pools. Extensive experiments show that OSMAS outperforms state-of-the-art baselines across five benchmarks, achieving higher performance with notably lower training costs. The code and trained pools are publicly available at <https://anonymous.4open.science/r/osmas-46F6>.

1 Introduction

Large Language Model (LLM)-based agents have demonstrated remarkable capabilities across diverse applications. Research has shown that multi-agent systems (MAS) organized through carefully designed workflows can achieve superior performance in various domains, ranging from software engineering (Qian et al., 2023) and scientific discovery (Boiko et al., 2023) to complex social simulation (Park et al., 2023). The emergence of MAS development frameworks, such as CAMEL (Li et al., 2023) and AutoGen (Wu et al., 2024) has significantly streamlined workflow design and prompt engineering, lowering the barrier to MAS development.

However, designing an effective MAS manually is a complex process requiring careful orchestration of multiple interdependent components (Qian et al., 2024; Wu et al., 2024). Designers must first define each agent’s role and capabilities (profiling), then architect the interaction topology among agents to ensure coherent collaboration. Prompts must be carefully crafted, as small wording changes can cascade through the system (Verma et al., 2024; Zhou et al., 2025). These three dimensions—profile, topology, and prompts—are deeply coupled, making manual optimization labor-intensive and costly.

Recent efforts to automate MAS design have targeted specific components. For instance, GPTSwarm (Zhuge et al., 2024), G-Designer (Zhang et al., 2024a) and ARG-Designer (Li et al., 2025) specialize in optimizing the communication topology between agents, while PromptBreeder (Fernando et al., 2023), DsPy (Liu et al., 2023) and MIPRO (Opsahl-Ong et al., 2024) focus on the iterative refinement of agent prompts. Another strategy, exemplified by AgentVerse (Chen et al., 2024) and EvoAgent (Yuan et al., 2025), involves optimizing the inherent profiles of the agents themselves. Although effective, these approaches operate within a confined search space defined by their respective components. A significant evolution in this field is marked by systems such as ADAS (Hu et al., 2024) and AgentSquare (Shang et al., 2024), which represent entire agentic workflows as executable code-represented programs and leverage LLMs as meta-optimizers to navigate this vastly expanded design space. This paradigm shift necessitates a higher level of abstraction, leading to the concept of *Agentic Operators* introduced by AFlow (Zhang et al., 2024b) and EvoFlow (Zhang et al., 2025). These operators encapsulate core agentic operations and serve as the fundamental units for exploring the architectural possibilities of multi-agent systems.

However, prior work still suffers from a fun-

damental **efficiency–adaptivity dilemma**. Many approaches either optimize individual components in isolation or jointly tune the system via a meta-agent (Hu et al., 2024; Zhang et al., 2024b; Wang et al., 2025a), which struggles with the complex, coupled search space. Either way, the result is a *single fixed MAS* applied to all queries, ignoring real-world query diversity. Conversely, full re-optimization per query is flexible but computationally prohibitive and yields unstable designs. Consequently, obtaining a MAS that is both cost-efficient and query-adaptive remains an open challenge (Wang et al., 2024; Zhou et al., 2024).

Inspired by one-shot neural architecture search (NAS) (Guo et al., 2020; Luo et al., 2019), we draw an analogy between MAS construction and NAS: agents can be likened to weights in a supernet, while the connectivity of the *supernet* corresponds to the architecture of the MAS. Just as NAS supernets share parameters across architectures, functionally similar MAS should share agents and be constructed in a one-shot manner.

Based on this insight, we propose **OSMAS** (**One-Shot Multi-Agent System**), an automated framework that resolves the efficiency–adaptivity dilemma by converting MAS design from “re-search every time” to “build once, compose efficiently.” OSMAS consists of two stages: **Super MAS Training** and **Pool-Based MAS Generation**. During the training stage, we decouple the optimization of agent profiles, prompts, and topology, and iteratively refine them to construct two reusable pools: an *Agent Pool*, which contains a collection of agents, and a *Topology Pool*, which stores a set of (*query*, *topology*) pairs learned from training data. In the generation stage, given an input query, a lightweight orchestrator rapidly composes a tailored MAS by retrieving and adapting appropriate agents and architecture from the pre-optimized pools, achieving efficient adaptation without expensive online optimization. Our approach enables truly comprehensive and fully automated MAS design. A comparison with representative prior work is summarized in Table 1 (only one representative work of each type is included for comparison).

Our work makes the following contributions:

- We propose **OSMAS**, a one-shot MAS design framework that resolves the efficiency–adaptivity dilemma by decoupling offline optimization from online composition via reusable Agent and Topology Pools.

Table 1: Comparison of automation capabilities across methods.

Method	Profile	Prompt	Topology	Query Adapt.
EvoAgent	✓	✗	✗	✗
MIPRO	✗	✓	✗	✗
ARG-D	✗	✗	✓	✓
AFlow	✓	✓	✓	✗
OSMAS	✓	✓	✓	✓

- We introduce a decoupled training paradigm that separately optimizes agent profiles, prompts, and system topology through iterative textual gradient refinement and multi-objective evolutionary search.
- We empirically validate OSMAS across diverse domains, demonstrating state-of-the-art performance with significantly lower training costs compared to prior methods.

2 Related Work

Automated MAS Design. The automation of multi-agent system design has been approached from three primary optimization perspectives. One line of work focuses on *topology optimization*, aiming to discover optimal communication structures between agents (Zhuge et al., 2024; Li et al., 2025). Another direction, *prompt optimization*, iteratively refines agent prompts (Khatab et al., 2024; Fernando et al., 2023). A third category concentrates on *agent profiling*, optimizing inherent agent characteristics (Yuan et al., 2025).

More recent frameworks use LLMs as meta-optimizers over entire workflows, notably ADAS (Hu et al., 2024) and AgentSquare (Shang et al., 2024). To structure this expanded search space, AFlow (Zhang et al., 2024b) and EvoFlow (Zhang et al., 2025) introduce modular *Agentic Operators* as reusable building blocks for system architecture exploration.

Unlike prior work that either isolates individual components or relies on costly meta-agent optimization, OSMAS decouples profile, prompt, and topology optimization into structured phases, constructing reusable Agent and Topology Pools. This pool-based paradigm enables one-shot, query-adaptive MAS generation via lightweight retrieval and composition, eliminating expensive online re-optimization.

3 Preliminary

3.1 Search Space and Objective

General Search Space. For a general MAS design task, the search space consists of all possible configurations:

$$\mathcal{W} = \{(P, T) \mid P \in \mathcal{P}, T \in \mathcal{T}\}, \quad (1)$$

where $\mathcal{P}$ denotes the set of prompts and $\mathcal{T}$ is the set of admissible topologies specifying inter-agent connectivity.

OSMAS Search Space. In OSMAS, we follow the concept of agentic operators from prior work: each agent is represented as a subgraph associated with an operator type that governs its internal topological structure. Although this constrains the search space, predefined operations significantly improve search efficiency. Under this formulation, the search space simplifies to:

$$\mathcal{W} = \{(\mathcal{S}, A) \mid A \in \mathcal{A}\}, \quad (2)$$

where $\mathcal{S} = \{S(O, F, P) \mid O \in \mathcal{O}, F \in \mathcal{F}, P \in \mathcal{P}\}$ defines all possible agent subgraphs parameterized by operator type O , profile F , and prompt P , and $\mathcal{A}$ denotes the set of inter-subgraph connections.

The connectivity is accordingly decomposed into intra-subgraph connectivity (governed by operator types) and inter-subgraph connectivity:

$$\mathcal{T} = \mathcal{O} \cup \mathcal{A}. \quad (3)$$

Each node’s prompt is determined by its agent’s prompt, profile, and operator: $p_v = \phi(p_i, f_i, o_i)$ for node v in subgraph i , where $\phi(\cdot)$ is a template-based prompt generation function.

3.2 Problem Formulation

MAS Optimization In a given scenario, the agent profile is first optimized; once fixed, the remaining design reduces to a joint optimization of prompts and topology. Let $P(q, a; W)$ denote the performance of MAS W on query q and $C(q; W)$ its computational cost. The overall objective is:

$$\max_W \mathbb{E}_{(q,a) \sim \mathcal{D}} [P(q, a; W) - \lambda C(q; W)], \quad (4)$$

where $\mathcal{D}$ is the dataset distribution and λ balances performance and cost. Prior work such as AFlow formulates this as:

$$(P^*, T^*) = \operatorname{argmax}_{P \in \mathcal{P}, T \in \mathcal{T}} \mathbb{E}_{(q,a) \sim \mathcal{D}} [G(P, T)]. \quad (5)$$

However, prompts (natural language) and topology (structured connectivity) have fundamentally different modalities, making joint LLM-based optimization ill-posed. Inspired by the two-stage co-optimization paradigm in one-shot NAS, we reformulate this as a bi-level optimization:

$$\begin{aligned} T^* &= \operatorname{argmax}_{T \in \mathcal{T}} \mathbb{E}_{(q,a) \sim \mathcal{D}} [G(P^*, T)] \\ \text{s.t. } P^* &= \operatorname{argmax}_{P \in \mathcal{P}} \mathbb{E}_{(q,a) \sim \mathcal{D}} [G(P, T)] \end{aligned} \quad (6)$$

The prompts used in topology evolution directly inherit the results from prompt optimization.

MAS Generation In conventional approaches, a general multi-agent system is first learned from a training dataset $\mathcal{D} = \{(q^{(i)}, a^{(i)})\}_{i=1}^N$, yielding a fixed MAS applied to all queries:

$$\text{MAS}_{\text{gen}} = \mathcal{F}(\mathcal{D}), \quad a = \text{MAS}_{\text{gen}}(q). \quad (7)$$

However, such a one-size-fits-all design fails to adapt to query diversity. In contrast, our goal is to generate a query-specific MAS tailored to each input:

$$\text{MAS}_q = \mathcal{G}(q; \mathcal{D}), \quad a = \text{MAS}_q(q). \quad (8)$$

4 Methodology

4.1 Overview

The overall framework of OSMAS is illustrated in Figure 1. It consists of two main stages: **Super MAS Training** and **Pool-Based MAS Generation**.

In the training stage, we optimize agent profiles, prompts, and topology through a three-phase iterative procedure. (1) First, we employ a *feedback-driven self-refinement strategy* to optimize agent profiles, yielding an initial set of agents along with their scenario-specific profiles. (2) Second, we evolve prompts under a given topology using *textual gradient*-based optimization. (3) Third, with prompts fixed, we perform *query-specific multi-objective evolutionary optimization* over topology, selecting suitable operators for each agent and determining inter-agent connections. This yields a Pareto front of high-performing topologies, from which the best topology for each query can be identified. The complete process of super MAS training is shown in Algorithm 1.

We alternate between phases (2) and (3) until convergence or up to a pre-specified maximum number of iterations, resulting in a **Super MAS**

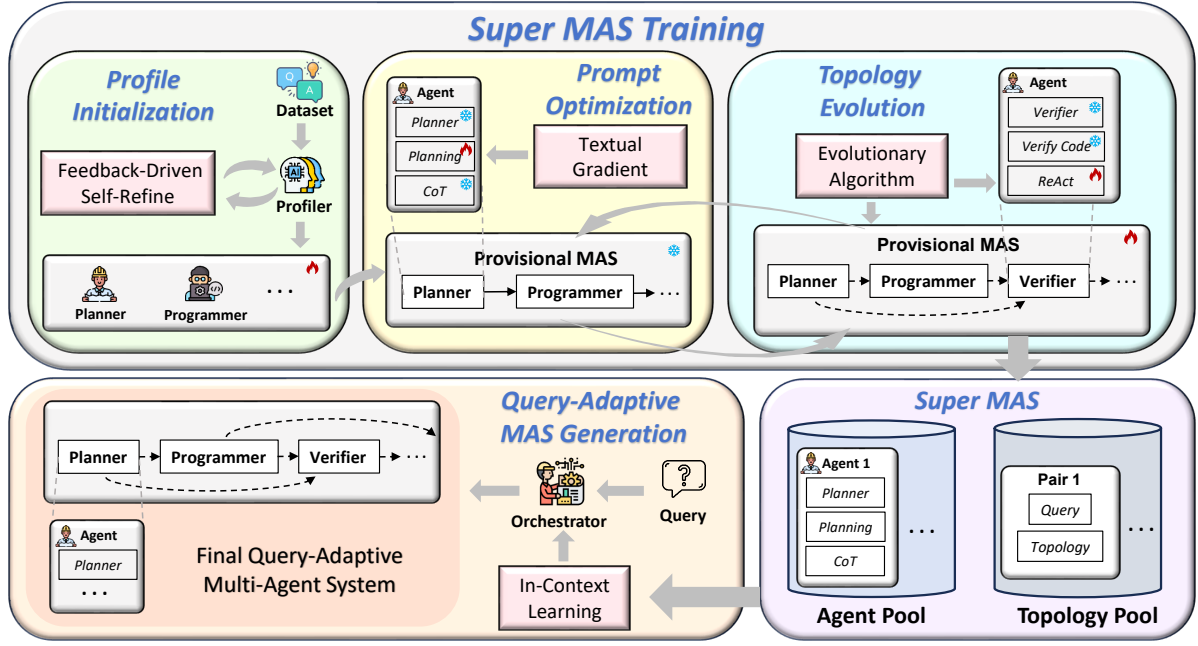


Figure 1: The overview of OSMAS.

consisting of a set of optimized agents and a collection of query-specific topologies. More precisely, OSMAS constructs two key pools from a limited set of (q, a) demonstrations: (i) an *Agent Pool* containing agents parameterized by their (profile, operator, prompt) triples; and (ii) a *Topology Pool* comprising (q, T) pairs, which serve as in-context demonstrations for topology generation.

In the generation stage, given a new query q , an orchestrator agent constructs a query-adaptive MAS by retrieving and adapting exemplars from the pre-optimized Agent and Topology Pools: agents are instantiated by customizing high-performing templates from the Agent Pool, while the topology is synthesized from similar (q, T) pairs in the Topology Pool. This eliminates the need for the LLM to generate a MAS from scratch; instead, it reuses components deeply optimized during Super MAS training, significantly reducing reasoning and context overhead while enabling both *fast* and *high-quality* generation.

4.2 Profile Initialization

To construct an MAS for a given task, the first step is to define the role of each agent. This involves using an LLM to specify an agent’s profile, including its persona and available tools, which determines its capabilities and thus its specific function within the system. However, relying solely on a meta-agent to generate multiple agent profiles remains chal-

lenging. Inspired by EvoAgent (Yuan et al., 2025), we conceptualize the agent generation procedure as an evolutionary process. Unlike EvoAgent, which focuses on evolving agents individually, we argue that agent profiles should be evaluated within an MAS and refined based on collective feedback.

Based on this, we propose a *Feedback-Driven Self-Refine Strategy*. Specifically, a profiler agent generates multiple agents based on the task description and a few examples. These agents are then organized into an simple MAS and evaluated on test cases. Based on the outcomes, including individual agent performance and the final evaluation feedback, the profiler agent is instructed to add or remove agents or refine the profiles of individual agents (lines 4 to 8 in Algorithm 1). This strategy ensures that the initial candidate agents are well-aligned with the task requirements. The details of this strategy are provided in the Appendix B.3.

4.3 Prompt Optimization

After the agent profiles are determined, their specific prompts require further optimization for optimal performance. Since these agents operate within an interdependent system, directly optimizing the entire system presents two major challenges: an excessively large optimization space and the difficult problem of credit assignment (Opsahl-Ong et al., 2024; Wang et al., 2025b). This makes many existing prompt optimization methods, such as MIPRO

Algorithm 1 Overall Workflow of Super MAS Training

```

1: Input: Dataset  $D$ , Candidate Models  $\mathcal{M}$ , Candidate Op-
   operators  $\mathcal{O}$ 
2: Output: Optimized Super MAS Pool  $P_{final}$ 
3: // Stage 1: Profiles Initialization
4:  $MAS \leftarrow \text{profiler}(D)$ 
5: repeat
6:    $feedback \leftarrow \text{evaluate}(MAS, D)$ 
7:    $MAS \leftarrow \text{profiler}(feedback)$ 
8: until  $\text{convergence}(Feedback)$ 
9: // Stage 2 & 3:
   // Iterative Prompt and Topology Co-optimization
10:  $P \leftarrow \text{metaAgent}(MAS, D)$ 
11: repeat
12:   // Stage 2: Prompt Tuning for current best MAS
13:    $MAS \leftarrow \text{getBest}(P)$ 
14:    $MAS \leftarrow \text{textGrad}(MAS, D)$ 
15:    $P.prompt \leftarrow MAS.prompt$ 
16:   // Stage 3: Topology Evolution
17:   repeat
18:     for  $(parent1, parent2) \in \text{select}(P)$  do
19:        $P_{children} \leftarrow \text{crossover}(parent1, parent2)$ 
20:        $P_{children} \leftarrow \text{mutate}(P_{children})$ 
21:     end for
22:      $P \leftarrow \text{update}(P \cup P_{children})$ 
23:   until  $\text{convergence}(P.topology)$ 
24: until  $\text{convergence}(P)$ 
25:  $P_{final} \leftarrow \text{getBest}(P)$ 
26: return  $P_{final}$ 

```

(Opsahl-Ong et al., 2024) and GEPA (Agrawal et al., 2025), less effective.

To address this, we adopt the *textual gradient-based approach* (Yuksekgonul et al., 2025) that jointly optimizes the prompts for all agents via backpropagation, thereby maximizing the final output quality. Specifically, given an MAS configuration and task loss, an LLM-based evaluator provides natural language feedback (i.e., textual gradients) on each agent’s prompt, which is then used by another LLM-based optimizer to generate an improved prompt. The formulation is provided in Appendix B.4.

This prompt optimization alternates with topology evolution. In the *first iteration*, the initial topology is set to a simple chain structure. Operators are initialized to the most basic direct generation, while tool-augmented agents are instantiated with the ReAct (Yao et al., 2022) operator. In *subsequent iterations*, the MAS is constructed using the best-performing topology from the current evolutionary population (line 13 in Algorithm 1). As the iterative optimization proceeds, each agent learns effective prompts tailored to diverse topologies through *textual gradient-based refinement* (line 14 in Algorithm 1). This process progressively enriches the Agent Pool with high-quality and topology-specific

configurations.

4.4 Topology Evolution

In this stage, we inherit the optimized agent profiles and prompts from the previous stages and focus solely on optimizing the internal operator of each agent and the connective communication between them, which governs the overall topology of the MAS. We designed a *customized evolutionary algorithm* (EA) (Eiben and Smith, 2015) to adjust the operators and inter-agent connections based on performance and cost. This algorithm can be formalized as follows.

Formulation. The agent topology is modeled as a Directed Acyclic Graph (DAG). Since operators can incorporate *iterative* processes like reflection, the unidirectional nature of DAG structure does not compromise the quality of the generation. This simplification offers key benefits: an agent’s internal structure, determined by the agent’s operator type, handles a specific task with its context isolated from other agents. Furthermore, it significantly reduces the search space and effectively leverages pre-defined, high-quality agentic operations.

Initialization. For the *first iteration* of topology evolution, the MAS population is initialized by a meta agent, as illustrated in line 10 of Algorithm 1. Specifically, agents that require tool use are directly assigned their operator type to ReAct (Yao et al., 2022), enabling natural tool invocation. For instance, a Verifier agent in a programming task, whose profile specifies the ability to “run code”, is initialized with a ReAct structure to interact iteratively with the tool. For other agents, the meta agent assigns the most suitable operator type and base model as initialization. The topology of the entire MAS is initialized with a simple chain.

Subsequent Iterations. In *subsequent iterations*, the population of topologies is carried over from the previous evolutionary generation. Critically, the operator type of each agent and the connection remain unchanged, while only their prompts are updated using the optimized prompts obtained from Stage 2 (line 15 in Algorithm 1).

Crossover. Intuitively, components common to both parent solutions are likely superior and should be preserved. For the connective topology, we employ a *Constraint-Preserving Edge Recombination Crossover*. The edge recombination crossover method (Whitley et al., 1989), proven effective for

graph-based problems, inherits edges common to both parent graphs and randomly selects the source parent for non-common edges. At the same time, we ensure the offspring remaining a valid DAG by adding constraints during the inheritance process. For operator types, the offspring inherits the choice if both parents agree; otherwise, one parent’s choice is selected randomly (line 19 in Algorithm 1).

Mutation. For the topology, we introduce a *Topology-Safe Local Reconnection Mutation*. This operation involves either adding or removing a connection locally. Added connections are restricted to point forward to subsequent agents, followed by a legitimization step to ensure the graph remains a DAG. Specifically, if a mutation leaves an agent with inputs but no outputs, an output connection to a subsequent agent is added. Conversely, if an agent is left with an output but no input, the mutation is reverted. For operator types and model selections, a single agent is chosen randomly for mutation (line 20 in Algorithm 1).

Selection and Population Update. Following these operations, a population of topology candidates is generated. Each candidate system is evaluated by running it on queries from a dataset, measuring both the result quality and computational cost. The candidates are ranked based on the optimization objective (Equation 4), and poorer-performing ones are discarded. The higher-ranked candidates are retained in the population to serve as parents for the next iteration (line 22 in Algorithm 1).

Query-Specific Objective. To construct the Topology Pool, we formulate topology evolution as a *query-specific multi-objective evolutionary problem*. Specifically, each training query $q^{(k)} \in \mathcal{Q}_{\text{train}}$ is treated as an independent optimization objective. The goal is to discover a diverse set of topologies that collectively achieve high performance across all queries, which naturally leads to a vector-valued optimization formulation. This is formalized as follows:

$$f^{(k)}(T) = G(T; q^{(k)}) - \lambda \cdot C(T; q^{(k)}), \quad (9)$$

$$\mathbf{F}(T) = (f^{(1)}(T), f^{(2)}(T), \dots, f^{(K)}(T))^{\top} \in \mathbb{R}^K, \quad (10)$$

where $G(T; q^{(k)})$ and $C(T; q^{(k)})$ denote the task performance and computational cost of topology T on query $q^{(k)}$, respectively, $\lambda \geq 0$ balances the

trade-off, and $K = |\mathcal{Q}_{\text{train}}|$ is the number of training queries.

We evolve a population of topologies $\mathcal{P}_{\text{pop}} \subset \mathcal{T}$ using a SOTA high-dimensional multi-objective optimizer NSGA-III (Deb and Jain, 2013) to approximate the Pareto front of $\mathbf{F}(T)$. Each non-dominated solution T^* in the final population is paired with the query $q^{(k)}$ for which it achieves the highest marginal gain, yielding a set of exemplar $(q^{(k)}, T^{(k)})$ pairs. These pairs are stored in the Topology Pool, enabling effective in-context retrieval during query-adaptive MAS generation.

In our customized evolutionary algorithm, the LLM is used solely for agent execution, not as an optimizer or for generating mutations, which substantially reduces costs. The details of the algorithm are provided in the Appendix B.5.

4.5 Super MAS

The customized evolutionary algorithm yields a population of high-performing MAS configurations. From this population, the inter-agent connections associated with each query are extracted and stored into the *Topology Pool*. Concurrently, the operators combined with the agent profiles and prompts obtained from Stage 1 (profile initialization) and Stage 2 (prompt optimization) form complete agent definitions, which are added to the *Agent Pool*. As a result, we obtain two structured repositories:

- **Agent Pool** $\mathcal{P}_{\text{agent}} = \{S_i\}_{i=1}^M$, where each subgraph S_i encapsulates a complete agent specification, including its profile, prompt, and operator;
- **Topology Pool** $\mathcal{P}_{\text{sys}} = \{(q^{(k)}, T^{(k)})\}_{k=1}^K$, which records, for each training query $q^{(k)}$, the corresponding near-optimal topology $T^{(k)}$ discovered during evolution.

Together, these two pools constitute the **Super MAS**—a reusable, pre-optimized design space that enables rapid, *pool-based MAS generation* across diverse downstream tasks.

4.6 Pool-Based MAS Generation

In OSMAS, $\mathcal{G}(q; \mathcal{D})$ (Equation 8) is instantiated through a composition mechanism grounded in two pre-constructed pools obtained during the Super MAS training stage. Given a new query q , the LLM-based orchestrator agent constructs MAS_q by retrieving agent templates from $\mathcal{P}_{\text{agent}}$ and topology

exemplars from $\mathcal{P}_{\text{sys}}$, then adapting them to the query context. Formally, the generated MAS is:

$$\text{MAS}_q = (\mathcal{S}(q; \mathcal{P}_{\text{agent}}), T(q; \mathcal{P}_{\text{sys}})), \quad (11)$$

where $\mathcal{S}(q; \mathcal{P}_{\text{agent}})$ denotes the set of query-adapted agents synthesized using exemplars from the Agent Pool, and $T(q; \mathcal{P}_{\text{sys}})$ denotes the topology synthesized from the Topology Pool.

Crucially, because both components are composed from offline-optimized pools rather than generated from scratch, the approach significantly improves generation reliability. This paradigm of leveraging offline-optimized pools for online composition enables OSMAS to achieve *one-shot, pool-based* MAS generation with minimal inference overhead. The detailed prompts used in this step is provided in the Appendix C.1.

5 Experiments

5.1 Experimental Setup

Datasets. We evaluate OSMAS on five public benchmarks spanning four domains: (1) **Mathematical reasoning:** GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021) (2) **Code generation:** HumanEval (Chen, 2021) and MBPP (Austin et al., 2021); (3) **Multi-hop QA:** HotpotQA (Yang et al., 2018). For the MATH dataset, we follow (Zhang et al., 2024b) in selecting 617 problems from four typical problem types (Combinatorics & Probability, Number Theory, Pre-algebra, Pre-calculus) at difficulty level 5. For HotpotQA, we randomly sample 1,000 examples.

We adhere to the official train/test splits provided with the benchmark. If no official split exists, we partition the data into validation and test sets at a 1:4 ratio. To reduce training costs, we randomly sample 20% of the training split for each benchmark during the training phase, rather than the full training set.

Baselines. We compare OSMAS against three categories of baselines:

(1) **Manually designed workflows:** IO (direct LLM call), Chain-of-Thought (CoT) (Wei et al., 2022), Self-Consistency CoT (5 answers) (Wang et al., 2022), MultiPersona Debate (Wang et al., 2024), Self-Refine (max 3 rounds) (Madaan et al., 2023), and LLM-Debate (Du et al., 2023).

(2) **Component-wise automated optimization methods:** These approaches optimize a single aspect of MAS design. GEPA (Agrawal

et al., 2025) focuses on prompt evolution, while GPTSwarm (Zhuge et al., 2024) specializes in communication topology discovery.

(3) **Automated MAS design methods:** ADAS (Hu et al., 2024), AgentSquare (Shang et al., 2024), and AFlow (Zhang et al., 2024b) — the strongest representative that jointly optimizes profiles, prompts, and topology.

Implementation Details. OSMAS uses a dual-model strategy: gpt-4o as the optimizer for prompt refinement during Super MAS training, and gpt-4o-mini as executor models for topology evolution and agent inference. All models are accessed via official APIs. In our implementation, we consider six operator types, which are detailed in Appendix B.1. The topology evolution is built using NSGA-III (Deb and Jain, 2013) and prompt optimization uses TextGrad (Yuksekgonul et al., 2024) with 3 gradient steps per iteration.

Metrics. We report benchmark-specific performance metrics: **Solve Rate (%)** for GSM8K and MATH; **pass@1** for HumanEval and MBPP; **F1 Score** for HotpotQA. Additionally, we track **inference cost** (USD) based on official API pricing across input and output tokens for all LLM calls.

5.2 Performance Analysis

Table 2 presents the performance of OSMAS and all baselines across the five benchmarks and their average. OSMAS achieves state-of-the-art results on all benchmarks and the highest average score, demonstrating its effectiveness in generating high-performing, task-specific MAS configurations.

5.3 Cost Analysis

Figure 2 illustrates three cost aspects on MATH to validate OSMAS’s efficiency: (1) inference cost of the generated MAS, (2) training cost of Super MAS, and (3) cost of pool-based MAS generation. All costs are normalized per query (USD).

Cost of Generated MAS Execution. Because OSMAS tailors the MAS topology and agent set to each query, it avoids unnecessary agent invocations present in fixed-topology baselines. So OSMAS achieves comparable inference cost with AFlow while delivering higher performance.

Cost of Super MAS Training. Although OSMAS trains a Super MAS (Agent Pool + Topology Pool) rather than a single MAS, its decoupled optimization and use LLMs only for prompt refine-

Table 2: Performance comparison across benchmarks. Best results are bolded.

Benchmark	IO	CoT	SC	MP	Debate	Refine	GEPA	Swarm	ADAS	AFlow	OSMAS
GSM8K	87.5	87.1	87.6	87.5	89.5	89.6	88.0	89.1	86.1	91.2	92.5
MATH	48.6	48.8	50.4	50.8	48.5	46.1	53.3	47.9	35.4	56.2	58.9
MBPP	71.8	71.8	73.6	73.6	70.3	69.8	65.5	77.4	53.4	83.4	85.3
HumanEval	87.0	88.6	91.6	89.3	88.7	87.8	85.6	89.3	82.4	94.7	95.5
HotpotQA	68.1	67.9	68.9	69.2	62.0	60.8	69.0	65.8	64.5	73.5	76.6
Avg.	72.6	72.8	74.4	74.1	71.8	70.8	72.3	73.9	64.4	79.8	81.8

Table 3: Ablation study

Variant	MATH
OSMAS (Full)	58.9
– Query Adaptivity	58.6
– Topology Evolution	57.0
– Prompt Optimization	52.3
– Profile Initialization	48.6

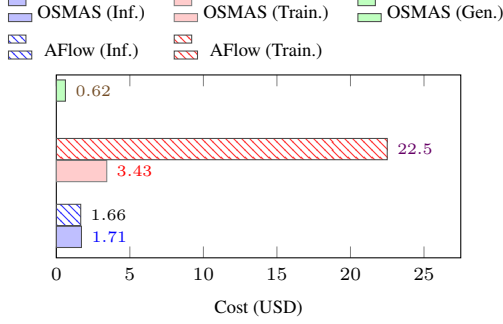


Figure 2: Cost analysis on MATH. Bars from top to bottom: (1) Inference cost, (2) Training cost, (3) OSMAS generation cost. All values in USD.

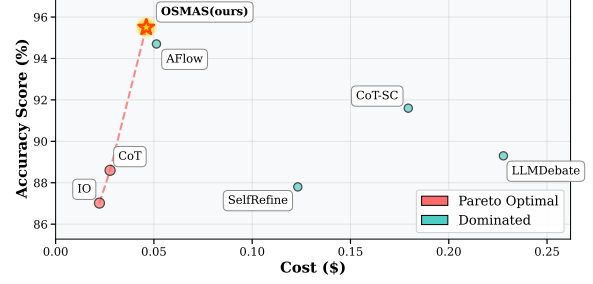


Figure 3: Performance–cost Pareto front on HumanEval. OSMAS achieves the best trade-off, lying on the Pareto frontier.

ment (not topology search), which keeps training costs modest. Despite achieving comparable design space exploration, the training cost of OSMAS is only 15.4% of that required by AFlow.

Cost of Pool-Based MAS Generation. The online generation phase requires a simple orchestrator agent to assemble the MAS from pre-optimized pools. This cost is minimal (averaging \$0.0013 per query), which is only 37.3% of the total inference cost and negligible compared to MAS execution.

5.4 Performance–Cost Trade-off

To further demonstrate the efficiency of OSMAS, we plot the performance–cost Pareto front across all methods on the HumanEval benchmark in Figure 3. Each point represents a method’s average solve rate (higher is better) versus its total inference cost (lower is better). OSMAS lies on the Pareto frontier, achieving the highest performance at a given cost level and the lowest cost at a given performance level.

5.5 Ablation Study

To evaluate the contribution of each component in OSMAS, we conduct ablation studies on MATH by removing: (1) query-level adaptivity (using a fixed MAS from the Super MAS Pool), (2) topology evolution (using a simple chain structure as topology), (3) prompt optimization (using only initial agent

profiles without prompt refinement), and (4) profile initialization (generating agents directly without defining agent profiles). Results in Table 3 show that each component is essential—removing any leads to a clear performance drop. Notably, profile initialization and prompt optimization are the most essential, while query adaptivity provides a smaller but consistent improvement.

6 Conclusion

In this work, we presented OSMAS, a one-shot framework for multi-agent system design that resolves the efficiency–adaptivity dilemma through a decoupled, pool-based paradigm. By separately optimizing agent profiles, prompts, and topology during an offline training phase, OSMAS constructs a reusable Super MAS consisting of an Agent Pool and a Topology Pool. At inference time, a lightweight orchestrator composes a task-specific MAS by retrieving and adapting from these pre-optimized pools, eliminating expensive online optimization while maintaining effective adaptation. Our experiments demonstrate that OSMAS achieves state-of-the-art performance across diverse benchmarks with significantly lower training costs compared to existing methods. We believe OSMAS provides a practical and scalable pathway toward deploying efficient, adaptive MAS in real-world applications.

642 Limitations

643 Our work has several limitations that warrant future
 644 investigation. First, OSMAS is evaluated exclu-
 645 sively with GPT-4o and GPT-4o-mini as backbone
 646 models; its transferability to open-source or smaller
 647 LLMs remains unexplored. Second, all bench-
 648 marks are in English and span only four domains
 649 (mathematical reasoning, code generation, multi-
 650 hop QA), leaving generalization to other languages,
 651 low-resource settings, and broader task types (e.g.,
 652 creative generation, multimodal reasoning) as open
 653 questions. Third, while our operator-based de-
 654 sign incorporates iterative patterns such as self-
 655 refinement within agent subgraphs, the inter-agent
 656 topology is still constrained to directed acyclic
 657 graphs, which may limit certain complex cross-
 658 agent interaction patterns. Fourth, although OS-
 659 MAS reduces training cost relative to prior work,
 660 the Super MAS construction phase still requires
 661 non-trivial LLM API calls, which may limit acces-
 662 sibility for resource-constrained researchers. Fifth,
 663 we train on only 20% of the available training
 664 data; how pool quality and downstream perfor-
 665 mance scale with training set size is not system-
 666 atically studied. Finally, the pool-based generation
 667 stage depends on an LLM orchestrator to retrieve
 668 and adapt exemplars, and its failure modes (e.g.,
 669 poor retrieval or misadaptation) have not been thor-
 670 oughly characterized.

671 Ethical Considerations

672 OSMAS automates the design of multi-agent sys-
 673 tems, which, while beneficial for research and pro-
 674 ductivity, could be misused to build agent pipelines
 675 for harmful purposes such as scalable disinforma-
 676 tion generation or automated social manipulation.
 677 Since OSMAS inherits the biases of its underlying
 678 LLMs, deploying it in sensitive domains without
 679 bias auditing carries fairness risks. We report all
 680 training costs and acknowledge the environmental
 681 footprint of repeated LLM API calls. Our eval-
 682 uation uses only established public benchmarks
 683 with no personal data, thus raising no privacy con-
 684 cerns. We release our code publicly to promote
 685 reproducibility and encourage future work on bias
 686 mitigation and multilingual evaluation.

687 References

688 Lakshya A Agrawal, Shangyin Tan, Dilara Soylu,
 689 Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Ar-

nav Singhvi, Herumb Shandilya, Michael J Ryan,
 Meng Jiang, and 1 others. 2025. Gepa: Reflec-
 tive prompt evolution can outperform reinforcement
 learning. *arXiv preprint arXiv:2507.19457*.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten
 Bosma, Henryk Michalewski, David Dohan, Ellen
 Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1
 others. 2021. Program synthesis with large language
 models. *arXiv preprint arXiv:2108.07732*.

Daniil A Boiko, Robert MacKnight, Ben Kline, and
 Gabe Gomes. 2023. Autonomous chemical research
 with large language models. *Nature*, 624(7992):570–
 578.

Mark Chen. 2021. Evaluating large language models
 trained on code. *arXiv preprint arXiv:2107.03374*.

Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang,
 Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu,
 Yi-Hsin Hung, Chen Qian, and 1 others. 2024. Agent-
 verse: Facilitating multi-agent collaboration and ex-
 ploring emergent behaviors. In *ICLR*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian,
 Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias
 Plappert, Jerry Tworek, Jacob Hilton, Reiichiro
 Nakano, and 1 others. 2021. Training verifiers
 to solve math word problems. *arXiv preprint
 arXiv:2110.14168*.

Kalyanmoy Deb and Himanshu Jain. 2013. An evo-
 lutionary many-objective optimization algorithm us-
 ing reference-point-based nondominated sorting ap-
 proach, part i: solving problems with box con-
 straints. *IEEE transactions on evolutionary com-
 putation*, 18(4):577–601.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenen-
 baum, and Igor Mordatch. 2023. Improving factual-
 ity and reasoning in language models through multi-
 agent debate. In *Forty-first International Conference
 on Machine Learning*.

Agoston E Eiben and James E Smith. 2015. *Introduc-
 tion to evolutionary computing*. Springer.

Chrisantha Fernando, Dylan Banarse, Henryk
 Michalewski, Simon Osindero, and Tim Rock-
 täschel. 2023. Promptbreeder: Self-referential
 self-improvement via prompt evolution. *arXiv
 preprint arXiv:2309.16797*.

Zichao Guo, Xiangyu Zhang, Haoyuan Mu, Wen Heng,
 Zechun Liu, Yichen Wei, and Jian Sun. 2020. Single
 path one-shot neural architecture search with uniform
 sampling. In *European Conference on Computer
 Vision*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul
 Arora, Steven Basart, Eric Tang, Dawn Song, and Ja-
 cob Steinhardt. 2021. Measuring mathematical prob-
 lem solving with the math dataset. *arXiv preprint
 arXiv:2103.03874*.

744	Shengran Hu, Cong Lu, and Jeff Clune. 2024. Automated design of agentic systems. <i>arXiv preprint arXiv:2408.08435</i> .	799
745		800
746		801
747	Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, Heather Miller, and 1 others. 2024. Dspy: Compiling declarative language model calls into state-of-the-art pipelines. In <i>The Twelfth International Conference on Learning Representations</i> .	802
748		803
749		804
750		805
751		806
752		
753		807
754	Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for "mind" exploration of large language model society. <i>Advances in Neural Information Processing Systems</i> , 36:51991–52008.	808
755		809
756		810
757		811
758		
759	Shiyuan Li, Yixin Liu, Qingsong Wen, Chengqi Zhang, and Shirui Pan. 2025. Assemble your crew: Automatic multi-agent communication topology design via autoregressive graph generation. <i>arXiv preprint arXiv:2507.18224</i> .	812
760		813
761		814
762		815
763		816
764	Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. 2023. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization. <i>arXiv preprint arXiv:2310.02170</i> .	817
765		818
766		819
767		820
768		821
769	Renqian Luo, Tao Qin, and Enhong Chen. 2019. Understanding and improving one-shot neural architecture optimization. <i>arXiv preprint arXiv:1909.10815</i> .	822
770		823
771		824
772	Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. <i>Advances in Neural Information Processing Systems</i> , 36:46534–46594.	825
773		826
774		827
775		
776		828
777	Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing instructions and demonstrations for multi-stage language model programs. <i>arXiv preprint arXiv:2406.11695</i> .	829
778		830
779		831
780		832
781		833
782	Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In <i>Proceedings of the 36th annual acm symposium on user interface software and technology</i> , pages 1–22.	834
783		835
784		836
785		837
786		838
787		
788	Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. 2023. Communicative agents for software development. <i>arXiv preprint arXiv:2307.07924</i> , 6(3):1.	839
789		840
790		841
791		842
792	Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, and 1 others. 2024. Chatdev: Communicative agents for software development. In <i>Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)</i> , pages 15174–15186.	843
793		844
794		845
795		846
796		847
797		848
798		849
		850
		851
		852
		853
		854
	Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. 2024. Agentsquare: Automatic llm agent search in modular design space. <i>arXiv preprint arXiv:2410.06153</i> .	
	Mudit Verma, Siddhant Bhambri, and Subbarao Kambhampati. 2024. On the brittle foundations of react prompting for agentic large language models. <i>arXiv preprint arXiv:2405.13966</i> .	
	Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. 2024. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. <i>arXiv preprint arXiv:2401.16158</i> .	
	Kun Wang, Guibin Zhang, ManKit Ye, Xinyu Deng, Dongxia Wang, Xiaobin Hu, Jinyang Guo, Yang Liu, and Yufei Guo. 2025a. Mas ² : Self-generative, self-configuring, self-rectifying multi-agent systems. <i>arXiv preprint arXiv:2509.24323</i> .	
	Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. <i>arXiv preprint arXiv:2203.11171</i> .	
	Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. 2025b. Evoagentx: An automated framework for evolving agentic workflows. In <i>Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations</i> , pages 643–655.	
	Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. <i>Advances in neural information processing systems</i> , 35:24824–24837.	
	L Darrell Whitley, Timothy Starkweather, and D’Ann Fuquay. 1989. Scheduling problems and traveling salesmen: The genetic edge recombination operator. In <i>Proceedings of the 3rd international conference on genetic algorithms</i> , pages 133–140.	
	Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, and 1 others. 2024. Autogen: Enabling next-gen llm applications via multi-agent conversations. In <i>First Conference on Language Modeling</i> .	
	Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In <i>Proceedings of the 2018 conference on empirical methods in natural language processing</i> , pages 2369–2380.	
	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language	

models. In *The eleventh international conference on learning representations*.

Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. 2025. Evoagent: Towards automatic multi-agent generation via evolutionary algorithms. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6192–6217.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. Textgrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639(8055):609–616.

Guibin Zhang, Kaijie Chen, Guancheng Wan, Heng Chang, Hong Cheng, Kun Wang, Shuyue Hu, and Lei Bai. 2025. Evoflow: Evolving diverse agentic workflows on the fly. *arXiv preprint arXiv:2502.07373*.

Guibin Zhang, Yanwei Yue, Xiangguo Sun, Guancheng Wan, Miao Yu, Junfeng Fang, Kun Wang, Tianlong Chen, and Dawei Cheng. 2024a. G-designer: Architecting multi-agent communication topologies via graph neural networks. *arXiv preprint arXiv:2410.11782*.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, and 1 others. 2024b. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*.

Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Serkan Ö Arık. 2025. Multi-agent design: Optimizing agents with better prompts and topologies. *arXiv preprint arXiv:2502.02533*.

Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, and 1 others. 2024. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294*.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*.

A Notations

Table 4 summarizes the key notations used throughout the paper.

Table 4: Summary of Notations.

Symbol	Description
$\mathcal{W}$	Search space of MAS configurations
$\mathcal{N}$	Set of possible agent nodes
$\mathcal{T}$	Set of admissible topologies
P	Set of prompts
O, F	Sets of operators and agent profiles
$S = (O, F, P)$	Agent subgraph specification
A	Set of inter-subgraph connections
$T = O \cup A$	Full MAS topology (intra + inter)
$P(\cdot)$	Task evaluation function
$C(\cdot)$	Inference cost (USD)
λ	Cost-performance trade-off coefficient
$\phi(\cdot)$	Prompt generation function
P_{agent}	Agent Pool
P_{sys}	System Design Pool
MAS_q	Query-adaptive MAS for query q

B Technical Details

B.1 Operator Space

We instantiate each agent as a modular operator following the design philosophy of AFlow (Zhang et al., 2024b) and EvoFlow (Zhang et al., 2025). The supported operators are:

- **Direct:** A single LLM call without structured reasoning.
- **CoT (Chain-of-Thought):** Step-by-step reasoning (Wei et al., 2022).
- **ReAct:** Interleaved reasoning and tool use (e.g., code execution) (Yao et al., 2022). The ReAct operator is always equipped with a tool, such as ‘code execution’ in program verification.
- **Multi-CoT:** Parallel CoT paths with ensemble voting.
- **SelfRefine:** Iterative self-correction based on internal feedback (Madaan et al., 2023).
- **Self-Consistency-Ensemble:** Multiple CoT paths with majority voting (Wang et al., 2022).

B.2 Implementation Details

Our system is built on **LangGraph**¹, a framework for building stateful multi-agent workflows. Each operator is instantiated as a *subgraph*—a self-contained node with internal logic and I/O interfaces.

Prompt optimization leverages **TextGrad** (Yuksekgonul et al., 2024), a black-box optimization framework that uses natural language as gradients. We treat each agent as a *BlackboxLLM* module, where forward passes produce outputs and backward passes generate textual feedback for prompt refinement. TextGrad is available at: <https://textgrad.com/index.html>.

Profile initialization follows the evolutionary paradigm of **EvoAgent** (Yuan et al., 2025), which generates diverse agent roles via mutation and selection. We adapt their feedback-driven self-refinement strategy to the MAS scenario, ensuring that agents are well-aligned with task requirements. EvoAgent details: <https://evo-agent.github.io/>.

B.3 Details of Profile Initialization

As outlined in Algorithm 2, profile optimization begins with a task description and few-shot examples. A *profiler agent* generates an initial set of candidate agents (e.g., “Planner”, “Verifier”). These agents are assembled into a provisional MAS and evaluated on validation queries. Based on performance feedback (both individual and collective), the profiler iteratively refines agent profiles until convergence. This ensures that the final Agent Pool contains only high-performing, task-relevant agents.

Algorithm 2 Profiles Optimization

```

1: Input: Dataset  $D$ 
2: Output: Optimized profiles
3: // Stage 1: Profiles Optimization
4:  $task, examples \leftarrow \text{loadData}(D)$ 
5:  $profiles \leftarrow \text{profiler.initialize}(task, examples)$ 
6: repeat
7:    $MAS_p \leftarrow \text{constructMAS}(agentProfiles)$ 
8:    $feedback \leftarrow \text{evaluate}(MAS_p, examples)$ 
9:    $reflection \leftarrow \text{profiler.reflect}(profiles, feedback)$ 
10:   $candidates \leftarrow \text{profiler.generate}(reflection)$ 
11:   $profiles \leftarrow \text{profiler.refineAndSelect}(candidates)$ 
12: until convergence( $feedback$ )
13: return  $profiles$ 

```

B.4 Textual Gradient in Prompt Optimization

Formally, following TextGrad (Yuksekgonul et al., 2024), we model prompt optimization as automatic

¹<https://langchain-ai.github.io/langgraph/>

differentiation over a computation graph of the MAS. Let each agent’s prompt be a differentiable variable:

$$p_i = \text{Variable}(p_i^0, \text{role_desc}, \text{requires_grad} = \text{True}) \quad (12)$$

The forward pass executes the MAS to produce an answer a , and a task-specific loss $\mathcal{L}(q, a)$ is evaluated (e.g., by an LLM assessing answer correctness). In the backward pass, textual gradients are computed for each p_i via:

$$\nabla_{p_i} \mathcal{L} = \text{LLM_backward}(p_i, \text{context}, \mathcal{L}), \quad (13)$$

where LLM_backward is an LLM that generates natural language criticism explaining how to improve p_i to reduce $\mathcal{L}$. Finally, the prompt is updated by a textual gradient descent step:

$$p_i^{\text{new}} = \text{TGD_step}(p_i, \nabla_{p_i} \mathcal{L}), \quad (14)$$

where TGD_step is another LLM that revises p_i based on the feedback $\nabla_{p_i} \mathcal{L}$. This framework enables end-to-end, black-box optimization of prompts within the full MAS context without requiring parameter access or differentiable proxies.

In OSMAS, we apply textual gradient-based optimization not in isolation, but within the context of a fixed MAS structure, comprising a set of agents $\mathcal{A} = \{a_i\}$ and a given topology T . Starting from initial prompts $\{p_i^0\}$, we invoke the TextGrad procedure to jointly refine all agent prompts while preserving the underlying topology. Crucially, the forward execution and loss evaluation are performed over the *entire* MAS, ensuring that prompt updates account for inter-agent dependencies induced by T . The resulting optimized prompts $\{p_i^{\text{new}}\}$, together with their associated profiles and operators, are then added to the Agent Pool as high-quality, topology-aware configurations. This process enables the systematic construction of a diverse and performant agent pool, grounded in black-box, LLM-driven optimization. The overall procedure is outlined in Algorithm 3.

B.5 Details of Customized EA

Our topology evolution (Algorithm 4) employs NSGA-III (Deb and Jain, 2013) to perform many-objective Pareto optimization, where each training query $q^{(k)} \in \mathcal{Q}_{\text{train}}$ constitutes an independent objective. Specifically, for a candidate topology T , we define a K -dimensional objective vector:

$$\mathbf{F}(T) = (f^{(1)}(T), f^{(2)}(T), \dots, f^{(K)}(T))^{\top}, \quad (15)$$

Algorithm 3 Prompt Optimization via Textual Gradient

```

1: Input: A set of agents  $\mathcal{A} = \{a_i\}_{i=1}^n$ ; A topology  $T$ ; A
   evaluation function  $\mathcal{L}(\cdot)$ .
2: Output: Updated prompts  $\{p_i^{\text{new}}\}_{i=1}^n$  for all agents.
3: // Forward Pass: Execute MAS
4:  $\text{MAS} \leftarrow (\mathcal{A}, T)$ 
5:  $a \leftarrow \text{MAS}(q)$ 
6:  $\ell \leftarrow \mathcal{L}(q, a)$ 
7: // Backward Pass: Compute Textual Gradients
8: for each agent  $a_i \in \mathcal{A}$  do
9:    $\nabla_{p_i} \ell \leftarrow \text{LLM\_backward}(p_i, \text{context}_i, \ell)$ 
10: end for
11: // Optimization Step: Update Prompts
12: for each agent  $a_i \in \mathcal{A}$  do
13:    $p_i^{\text{new}} \leftarrow \text{TGD\_step}(p_i, \nabla_{p_i} \ell)$ 
14: end for
15: return  $\{p_i^{\text{new}}\}_{i=1}^n$ 

```

where $f^{(k)}(T) = P(\text{MAS}_T; q^{(k)}) - \lambda \cdot C(\text{MAS}_T; q^{(k)})$ measures the performance-cost trade-off on query $q^{(k)}$. NSGA-III maintains population diversity by associating solutions with a set of predefined reference points $\{\mathbf{r}_j\}$ on a normalized hyperplane, and selects individuals that best represent underpopulated reference directions while preserving non-dominated fronts.

The population consists of DAG-based topologies where:

- **Crossover** uses edge recombination (Whitley et al., 1989) with DAG constraints to ensure acyclicity;
- **Mutation** performs safe local rewiring (add/remove edges forward-only) to maintain valid execution order;
- **Selection** retains non-dominated solutions based on the multi-objective criterion (performance $- \lambda \cdot \text{cost}$) across all queries.

After convergence, the final non-dominated front $\mathcal{P}^*$ is used to construct the Topology Pool. For each query $q^{(k)}$, we select the topology $T^{(k)} \in \mathcal{P}^*$ that achieves the highest $f^{(k)}(T)$, forming the exemplar pair $(q^{(k)}, T^{(k)})$. The resulting set $\mathcal{P}_{\text{topo}} = \{(q^{(k)}, T^{(k)})\}_{k=1}^K$ —together with the pre-optimized Agent Pool $\mathcal{P}_{\text{agent}}$ —constitutes the Super MAS, enabling efficient query-adaptive MAS generation at inference time.

C Prompt Details

C.1 Prompts for MAS Generation

At inference time, an *orchestrator agent* constructs a query-specific MAS through a two-stage process.

Algorithm 4 Topology Evolution

```

1: Input: Dataset  $D$ , Agent Pool  $\mathcal{P}_{\text{agent}}$  (from Stage 1–2)
2: Output: Agent Pool  $\mathcal{P}_{\text{agent}}$  and Topology Pool  $\mathcal{P}_{\text{topo}}$  (i.e., Super MAS)
3:  $\text{task\_description}, \text{examples} \leftarrow \text{loadData}(D)$ 
4:  $\mathcal{P} \leftarrow \text{metaAgent}(P, \text{task}, \text{examples})$ 
5: repeat
6:   for  $(\text{parent1}, \text{parent2}) \in \text{select}(P)$  do
7:      $\text{child.edges} \leftarrow \text{crossoverEdges}(\text{parent1}, \text{parent2})$ 
8:      $\text{child.ops} \leftarrow \text{crossoverOp}(\text{parent1}, \text{parent2})$ 
9:      $\text{child.edges} \leftarrow \text{mutateEdges}(\text{child})$ 
10:     $\text{MAS} \leftarrow \text{constructMAS}(\mathcal{P}_{\text{agent}}, \text{child})$ 
11:    for  $q^{(k)} \in \mathcal{Q}_{\text{train}}$  do
12:       $f^{(k)} \leftarrow G(\text{MAS}; q^{(k)}) - \lambda C(\text{MAS}; q^{(k)})$ 
13:    end for
14:     $\mathbf{F}(\text{child}) \leftarrow (f^{(1)}, \dots, f^{(K)})^\top$ 
15:     $\mathcal{P}.\text{append}(\text{child})$ 
16:  end for
17:   $\mathcal{P} \leftarrow \text{NSGA-III-Update}(\mathcal{P}, \{\mathbf{F}(\text{ind})\}_{\text{ind} \in \mathcal{P}})$ 
18: until convergence( $\mathcal{P}$ ) or max iterations
19:  $\mathcal{P}^* \leftarrow \text{getParetoFront}(\mathcal{P})$ 
20:  $\mathcal{P}_{\text{topo}} \leftarrow \text{buildTopologyPool}(\mathcal{P}^*)$ 
21: return  $(\mathcal{P}_{\text{agent}}, \mathcal{P}_{\text{topo}})$ 

```

First, it retrieves relevant exemplars from both the Topology Pool (containing (q, T) pairs) and the Agent Pool (containing agent specifications), and synthesizes a high-level design of the MAS including the set of agents to use and their interaction topology, based on in-context learning. Second, it translates this design into a structured JSON format that can be directly parsed and executed by the LangGraph runtime. The orchestrator uses the following meta-prompt:

```

You are designing a multi-agent system
  → for the following query using in-
  → context learning from previous
  → successful systems.

**New Query:**
{query}

**Retrieved Successful Examples (from
  → MAS Pool):**
{examples}

**Important Principle: Simplicity First
  → **
- Start with the minimal number of
  → agents that can solve the problem
- 1-2 agents for simple/medium
  → complexity problems
- 3+ agents only if the problem clearly
  → requires specialized expertise
- Prefer linear sequential connections
  → over complex branching
- Each additional agent should add clear
  → , unique value

**Your Task:**
Based on the retrieved examples,
  → construct a context that will
  → guide the creation of an optimal

```

```

  → multi-agent system configuration.

**Output Format:**

**Successful Patterns Identified:**
1. Common agent roles across successful
  → systems
2. Effective operator choices for
  → different agent types
3. Connection patterns that work well
  → for similar problems
4. Key architectural insights from high-
  → performing systems

**Recommended System Architecture:**
Based on the patterns above, recommend:
- Agent descriptions (role and
  → responsibilities for each agent)
  → - **keep number minimal**
- Connection topology - **prefer
  → sequential for simpler problems**
- Operator assignments
- Reasoning for each recommendation

**Expected Challenges:**
- Potential difficulties based on query
  → characteristics
- How retrieved examples address similar
  → challenges
"""

```

```

**New Query:**
{query}

**System Context (Learned from
  → Successful Examples):**
{system_context}

**Your Task:**
Generate a complete, executable multi-
  → agent system configuration in
  → JSON format.

**Output JSON Format:**
```json
{{
 "analysis": {{
 "problem_type": "...",
 "complexity": "...",
 "num_agents": <number>
 }},
 "agents": [
 {{
 "name": "agent_name",
 "description": "Detailed
 → description of agent's role
 → and responsibilities...",
 "operator": "operator_name"
 }}
],
 "connections": [
 [<source_node>, <target_node>],
 ...
],
 "reasoning": "Explanation of why this
 → architecture is suitable..."
}}
```

```

```
1153 **Available Operators:**
1154 - generate: Generate solution directly
1155 - generate_cot: Generate solution with
1156   ↪ chain-of-thought
1157 - multi_generate_cot: Generate multiple
1158   ↪ candidates and pick best
1159 - self_refine: Generate and iteratively
1160   ↪ refine
1161 - test: Execute and test the solution
1162 - sc_ensemble: Self-consistency ensemble
1163
1164 **Node Numbering:**
1165 - 0: START
1166 - 1 to N: Agent nodes
1167 - N+1: END
1168
1169 Generate the complete JSON configuration
1170 ↪ now.
```

1172 This prompt enables reliable, one-shot generation
1173 of executable MAS while leveraging pre-optimized
1174 agent definitions, ensuring both correctness and
1175 efficiency.